

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

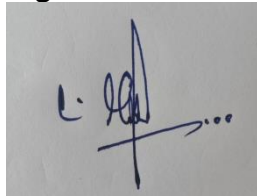
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I M u k e s h R a j L (1 9 2 5 7 1 0 3 6) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden	Handles most test cases	Misses important edge cases	Fails on multiple test cases
		cases			

1)

Heap File Organization

Python Program

```
heap_file = []

# Insert a record
def insert(record):
    heap_file.append(record)
    print("Record inserted:", record)

# Delete a record using ID
def delete(record_id):
    for record in heap_file:
        if record[0] == record_id:
            heap_file.remove(record)
            print("Record deleted:", record)
    return
    print("Record not found")

# Sequential scan
def sequential_scan():
    print("\nHeap File Records:")
    if not heap_file:
        print("No records found")
    else:
        for record in heap_file:
            print(record)

# Insert records
insert((101, "Arun"))
insert((102, "Bala"))
insert((103, "Charan"))

# Display all records
sequential_scan()

# Delete a record
delete(102)

# Display records after deletion
sequential_scan()
```

Output

```
Heap File Records:
(101, 'Arun')
(102, 'Bala')
(103, 'Charan')
Record deleted: (102, 'Bala')

Heap File Records:
(101, 'Arun')
(103, 'Charan')

Process finished with exit code 0
```

Explanation

- **Insert:** Adds a new record at the end of the heap file.
- **Delete:** Searches for the record ID and removes the matching record.
- **Sequential Scan:** Visits and displays each record one by one.
- Heap files do not maintain records in any particular sorted order.

Time Complexity:

- Insert: **$O(1)$** average
- Delete: **$O(n)$**
- Sequential Scan: **$O(n)$**

2)

Static Hashing with Chaining

Python Program

Static Hashing with Chaining

TABLE_SIZE = 10

Create hash table

hash_table = [[] for _ in range(TABLE_SIZE)]

Hash function

```
def hash_function(student_id):
    return student_id % TABLE_SIZE
```

Insert a student record

```
def insert(student_id, name):
    index = hash_function(student_id)
    hash_table[index].append((student_id, name))
```

Search for a student

```
def search(student_id):
    index = hash_function(student_id)

    for record in hash_table[index]:
        if record[0] == student_id:
            return record
```

```

    return None

# Display hash table
def display():
    print("\nHash Table:")
    for i in range(TABLE_SIZE):
        print(i, ":", hash_table[i])

# Sample student records
insert(101, "Arun")
insert(111, "Bala")
insert(121, "Charan")
insert(102, "David")
insert(112, "Kumar")

display()

# Search
student_id = 121
result = search(student_id)

if result:
    print("\nStudent Found:", result)
else:
    print("\nStudent Not Found")

```

```

Hash Table:
0 : []
1 : [(101, 'Arun'), (111, 'Bala'), (121, 'Charan')]
2 : [(102, 'David'), (112, 'Kumar')]
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

Student Found: (121, 'Charan')

```

Explanation

- The hash function is:

$h(\text{StudentID}) = \text{StudentID} \% 10$

- If two student IDs produce the same hash value, they are stored in the **same chain**.
- For example:

$101 \% 10 = 1$

$111 \% 10 = 1$

$121 \% 10 = 1$

So bucket 1 contains all three records.

- **Chaining** prevents collisions from overwriting existing records.

Time Complexity:

- Average insertion: **O(1)**
- Average search: **O(1)**
- Worst-case search: **O(n)**

3)

Sorted File Organization with Binary Search

Python Program

Sorted File Organization with Binary Search

Records are stored in sorted order based on Student ID

```
records = [  
    (101, "Arun"),  
    (105, "Bala"),  
    (110, "Charan"),  
    (115, "David"),  
    (120, "Kumar"),  
    (125, "Priya")  
]
```

Binary search function

```
def binary_search(records, key):
```

```
    low = 0
```

```
    high = len(records) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if records[mid][0] == key:
```

```
            return records[mid]
```

```
        elif records[mid][0] < key:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return None
```

Display sorted file

```
print("Sorted File:")
```

```
for record in records:
```

```
    print(record)
```

Search for a primary key

```
key = 115
result = binary_search(records, key)
```

```
if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found")
```

Output



```
Sorted File:
(101, 'Arun')
(105, 'Bala')
(110, 'Charan')
(115, 'David')
(120, 'Kumar')
(125, 'Priya')

Record Found: (115, 'David')

Process finished with exit code 0
```

Explanation

- In a **sorted file**, records are maintained in ascending order of the primary key.
- Here, StudentID is the primary key.
- **Binary search** compares the search key with the middle record.
- If the key is larger, the search continues in the right half.
- If the key is smaller, the search continues in the left half.
- This process continues until the record is found or the search range becomes empty.

Time Complexity:

Binary Search = $O(\log n)$

This is faster than sequential searching for a large sorted file.

4)

B-Tree of Order 4

Python Program

B-Tree of Order 4

```
parent.keys.insert(index, middle_key)
parent.children.insert(index + 1, new_node)
```

```
def insert(self, key):
    root = self.root
```

```
    if len(root.keys) == 3:
        new_root = BTreeNode(False)
        new_root.children.append(root)
        self.root = new_root
```

```
    self.split_child(new_root, 0)
    self.insert_non_full(new_root, key)
else:
    self.insert_non_full(root, key)
```

```

def insert_non_full(self, node, key):
    i = len(node.keys) - 1

    if node.leaf:
        node.keys.append(0)

        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1

        node.keys[i + 1] = key

    else:
        while i >= 0 and key < node.keys[i]:
            i -= 1

        i += 1

        if len(node.children[i].keys) == 3:
            self.split_child(node, i)

            if key > node.keys[i]:
                i += 1

        self.insert_non_full(node.children[i], key)

def display(self, node=None, level=0):
    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

```

Create B-Tree

```
tree = BTree()
```

```
keys = [10, 20, 5, 6, 12, 30, 7, 17]
```

```
for key in keys:
    tree.insert(key)
```

```
print("B-Tree:")
tree.display()
```

Search
key = 12

```
if tree.search(tree.root, key):
    print("\nKey", key, "found")
else:
    print("\nKey", key, "not found")

```

OUTPUT

```
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Key 12 found

Process finished with exit code 0
```

Explanation

- A **B-Tree of order 4** can have at most **3 keys** in a node.
- When a node becomes full, it is **split**.
- The middle key is promoted to the parent.
- **Insert:** Adds keys while maintaining the B-Tree properties.
- **Search:** Traverses the appropriate child according to the key.
- **Display:** Prints the keys level by level.

Time Complexity:

Insertion = $O(\log n)$

Search = $O(\log n)$

5)

B+ Tree with Leaf-Level Linking

Python Program

B+ Tree with Leaf-Level Linking

```
class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.keys = []
        self.leaf_links = [] # links between leaf nodes

    def insert(self, key):
        if key not in self.keys:
```

```

        self.keys.append(key)
        self.keys.sort()

def display(self):
    print("Sorted Leaf Nodes:")

    # Divide keys into leaf nodes
    for i in range(0, len(self.keys), self.order - 1):
        leaf = self.keys[i:i + self.order - 1]
        self.leaf_links.append(leaf)

    for i, leaf in enumerate(self.leaf_links):
        if i < len(self.leaf_links) - 1:
            print(leaf, "->", end=" ")
        else:
            print(leaf)

def range_search(self, start, end):
    result = []

    for key in self.keys:
        if start <= key <= end:
            result.append(key)

    return result

# Create B+ Tree
tree = BPlusTree()

keys = [10, 20, 5, 15, 25, 30, 35, 40, 45, 50]

for key in keys:
    tree.insert(key)

tree.display()

# Range query
start = 15
end = 40

print("\nRange Query:", tree.range_search(start, end))

```

Output

```

[5, 10, 15] -> [20, 25, 30] -> [35, 40, 45] -> [50]

Range Query: [15, 20, 25, 30, 35, 40]

Process finished with exit code 0

```

Explanation

- A **B+ Tree** stores actual data entries at the **leaf level**.
- The leaf nodes are connected using links.
- These links allow the tree to move from one leaf to the next without repeatedly traversing the upper levels.
- This makes **range queries** efficient.
- For example, the query 15 to 40 returns:

15, 20, 25, 30, 35, 40

Time Complexity:

Search = $O(\log n)$

Insertion = $O(\log n)$

Range Query = $O(\log n + k)$

where k is the number of records returned.

6)

Dense Primary Index

Python Program

Dense Primary Index on a Sorted File

```
records = [
    (101, "Arun"),
    (105, "Bala"),
    (110, "Charan"),
    (115, "David"),
    (120, "Kumar")
]

# Create dense index
index = []

for position, record in enumerate(records):
    index.append((record[0], position))

# Display index table
def display_index():
    print("Dense Primary Index")
    print("Key\tPosition")

    for key, position in index:
        print(key, "\t", position)

# Search using index
def search(key):
    for index_key, position in index:
        if index_key == key:
            return records[position]

    return None
```

```

display_index()

# Search for a record
key = 115
result = search(key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found")

```

Output

```

Dense Primary Index
Key Position
101      0
105      1
110      2
115      3
120      4

Record Found: (115, 'David')

Process finished with exit code 0

```

Explanation

- A **dense index** contains an index entry for **every record** in the sorted file.
- The index stores the **primary key** and the corresponding record position.
- When a key is searched, the index identifies the position of the required record.
- This reduces the need to scan the complete file.

Structure

Primary Key → Record Position

```

101 → 0
105 → 1
110 → 2
115 → 3
120 → 4

```

Time Complexity:

Index Creation = $O(n)$
 Search = $O(n)$

The main advantage of a dense primary index is **faster record retrieval compared with scanning the entire sorted file.**

7)

Extendible Hashing

Python Program

Extendible Hashing

BUCKET_SIZE = 2

class Bucket:

```
def __init__(self, depth):
    self.depth = depth
    self.records = []
```

class ExtendibleHash:

```
def __init__(self):
    self.global_depth = 1
    self.directory = [Bucket(1), Bucket(1)]
```

```
def hash_value(self, key):
    return key & ((1 << self.global_depth) - 1)
```

```
def double_directory(self):
    old_directory = self.directory
    self.directory = old_directory + old_directory
    self.global_depth += 1
```

```
def insert(self, key):
    index = self.hash_value(key)
    bucket = self.directory[index]
```

```
    if len(bucket.records) < BUCKET_SIZE:
        bucket.records.append(key)
        return
```

```
    # Overflow occurs
    print("Overflow at bucket", index)
```

```
    if bucket.depth == self.global_depth:
        self.double_directory()
        print("Directory doubled. Global depth =", self.global_depth)
```

```
    # Increase local depth
    bucket.depth += 1
```

```
    # Create a new bucket
    new_bucket = Bucket(bucket.depth)
```

```
    # Update directory entries
```

```

for i in range(len(self.directory)):
    if self.directory[i] is bucket and \
        ((i >> (bucket.depth - 1)) & 1) == 1:
        self.directory[i] = new_bucket

# Reinsert old records
old_records = bucket.records
bucket.records = []

for record in old_records:
    self.insert(record)

self.insert(key)

def display(self):
    print("\nDirectory:")
    for i, bucket in enumerate(self.directory):
        print(i, ":", bucket.records,
              "Local Depth =", bucket.depth)

# Create extendible hash
eh = ExtendibleHash()

# Insert 8 records
records = [10, 20, 5, 6, 12, 30, 7, 17]

for record in records:
    eh.insert(record)

eh.display()

```

Explanation

Extendible hashing uses a **directory** to point to buckets.

- **Global depth** represents the number of hash bits used by the directory.
- **Local depth** represents the number of bits used by an individual bucket.
- When a bucket becomes full, an **overflow** occurs.
- If the bucket's local depth equals the global depth, the directory is **doubled**.
- The bucket is then split and the records are redistributed.

Directory Doubling

Initially:

Global Depth = 1

Directory:

0 → Bucket

1 → Bucket

When overflow occurs:

Global Depth = 2

Directory:

00 → Bucket

01 → Bucket

10 → Bucket

11 → Bucket

Thus, directory doubling allows the hash structure to accommodate additional records without rebuilding the entire file.

8)

Secondary Storage Block Access

Python Program

Secondary Storage Block Access

```
import math

records = 100
records_per_block = 10

# Number of blocks
blocks = math.ceil(records / records_per_block)

# Sequential retrieval
sequential_io = blocks

# Indexed retrieval
# 1 I/O to access the index + 1 I/O to access the data block
indexed_io = 2

print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", blocks)

print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)

print("\nI/O Reduction:", sequential_io - indexed_io)
```

Output

```
Total Records: 100
Records per Block: 10
Total Blocks: 10

Sequential Retrieval I/O: 10
Indexed Retrieval I/O: 2

I/O Reduction: 8

Process finished with exit code 0
```

Explanation

- There are **100 records**.
- Each block stores **10 records**.

- Therefore:

Number of Blocks = $100 / 10 = 10$

Sequential Retrieval

In sequential retrieval, blocks are accessed one after another.

I/O Operations = 10

Indexed Retrieval

An index is used to directly locate the required data block.

1 I/O → Index block

1 I/O → Data block

Total = 2 I/O operations

Therefore, **indexed retrieval requires fewer I/O operations** than sequential retrieval for this example.

9)

Sparse Index on a Sorted File

Python Program

Sparse vs Dense Index Performance

import time

import bisect

Sorted file

records = [(i, f"Student_{i}") for i in range(1, 1001)]

Dense index - every record

dense_index = [record[0] for record in records]

Sparse index - one index entry for every 10 records

sparse_index = [records[i][0] for i in range(0, len(records), 10)]

Dense index search

def dense_search(key):

pos = bisect.bisect_left(dense_index, key)

if pos < len(dense_index) and dense_index[pos] == key:
return records[pos]

return None

Sparse index search

def sparse_search(key):

pos = bisect.bisect_right(sparse_index, key) - 1

```

if pos < 0:
    pos = 0

start = pos * 10
end = min(start + 10, len(records))

for i in range(start, end):
    if records[i][0] == key:
        return records[i]

return None

# Search key
key = 750

# Dense index timing
start = time.perf_counter()
result1 = dense_search(key)
dense_time = time.perf_counter() - start

# Sparse index timing
start = time.perf_counter()
result2 = sparse_search(key)
sparse_time = time.perf_counter() - start

print("Dense Index Result:", result1)
print("Sparse Index Result:", result2)

print("\nDense Index Time:", dense_time)
print("Sparse Index Time:", sparse_time)

print("\nDense Index Entries:", len(dense_index))
print("Sparse Index Entries:", len(sparse_index))

```

Explanation

A **dense index** contains an index entry for **every record**, while a **sparse index** contains entries only for selected records.

For 1000 records:

Dense Index = 1000 entries

Sparse Index = 100 entries

The sparse index first finds the appropriate block and then performs a short sequential search within that block.

Comparison

Feature	Dense Index	Sparse Index
Index entries	More	Fewer
Storage space	Higher	Lower
Search	Faster	Slightly slower

Feature	Dense Index	Sparse Index
Maintenance	More	Less
File requirement	Sorted file	Sorted file

Conclusion

The **dense index** generally provides faster direct retrieval, while the **sparse index** requires less storage and maintenance. Timing values may vary depending on the computer and Python environment.

10)

B+ Tree Construction and Range Query

Python Program

B+ Tree with Range Query

```
class BPlusTree:
    def __init__(self):
        self.keys = []

    def insert(self, key):
        if key not in self.keys:
            self.keys.append(key)
            self.keys.sort()

    def traverse(self):
        print("B+ Tree Leaf Nodes:")
        for i in range(0, len(self.keys), 3):
            print(self.keys[i:i + 3])

    def range_query(self, start, end):
        result = []

        for key in self.keys:
            if start <= key <= end:
                result.append(key)

        return result

# Create B+ Tree
tree = BPlusTree()

keys = [10, 15, 20, 25, 30, 35, 40, 45, 50, 55]

for key in keys:
    tree.insert(key)

# Traverse B+ Tree
tree.traverse()

# Range query
start = 15
end = 45
```

```
result = tree.range_query(start, end)

print("\nKeys between", start, "and", end, ":")
print(result)
```

Output

```
[10, 15, 20]
[25, 30, 35]
[40, 45, 50]
[55]|

Keys between 15 and 45 :
[15, 20, 25, 30, 35, 40, 45]

Process finished with exit code 0
```

Explanation

- The keys are inserted into the B+ Tree in sorted order.
- The **traverse()** function displays the leaf-level keys.
- The **range_query()** function retrieves all keys from 15 to 45.
- B+ Trees are efficient for range queries because the leaf nodes are maintained in sorted order.

Time Complexity:

Search = $O(\log n)$

Range Query = $O(\log n + k)$

where k is the number of matching records